

Trabalho 1 - Tabela Hash com CPFs

INF1010 - Estruturas de Dados Avancadas - 2026.1

PUC-Rio

Julia Kymie Dias Okada - Matricula: 2421312

15 de maio de 2026

1. Identificacao

Aluna: Julia Kymie Dias Okada
Matricula: 2421312
Disciplina: INF1010 - Estruturas de Dados Avancadas
Turma: 3VC / 3WA - 2026.1
Entrega: 15 de maio de 2026

2. Objetivo

O trabalho pede pra guardar cerca de 4096 CPFs numa tabela hash de tamanho fixo, usando endereçamento aberto pra resolver colisoes. A tabela tem que ter uma folga de uns 20% em relacao ao numero de chaves. Alem disso, o programa precisa gerar dados pra montar um grafico mostrando quantas colisoes foram acumulando conforme as chaves sao inseridas (de 100 em 100 ate 4096), pra dar pra ver o comportamento da funcao hash na pratica.

3. Estrutura do Programa

hash_cpf.c

inicializa()

preenche a tabela inteira com -1 (slot vazio).

insere(cpf)

insere o CPF usando double hashing; retorna quantas colisoes ocorreram antes de achar slot livre, ou -1 se a tabela estiver cheia.

main()

le CPFsValidos.txt, insere os CPFs acumulando colisoes e imprime uma linha CSV a cada 100 insercoes.

grafico.py

Le o CSV e plota o grafico de colisoes acumuladas com matplotlib. Salva em grafico_colisoes.png.

CPFValidos.txt

Arquivo de entrada fornecido pelo professor com 4096 CPFs validos, um por linha.

4. Solucao

4.1 Tamanho da tabela

Com 4096 chaves e folga de 20%, o minimo seria 4915. Foi escolhido $N = 5003$, que e o primeiro primo acima disso. Usar numero primo e importante pro double hashing funcionar direito, porque garante que a sondagem percorra todas as posicoes da tabela antes de repetir.

Fator de carga com todas as chaves: $4096 / 5003 = 81,9\%$

4.2 Funcao hash

```
h1(cpf) = cpf % 5003
```

O CPF e tratado como long long. O modulo por primo espalha bem as chaves porque nao tem padrao de

multiplo que criaria grupos de colisoes.

4.3 Tratamento de colisoes - Double Hashing

Foi usado double hashing porque e o que gera menos agrupamento entre os tres metodos de endereçamento aberto:

- Linear: simples, mas forma blocos grandes conforme enche.
- Quadratico: resolve o agrupamento primario mas ainda tem o secundario.
- Double hashing: cada chave tem seu proprio passo, entao nao agrupa.

A sondagem funciona assim:

```
pos(i) = (h1(cpf) + i * h2(cpf)) % N
h2(cpf) = 1 + (cpf % 4999)
```

O 4999 tambem e primo e ligeiramente menor que N. Isso garante que o passo nunca seja zero e que $\text{mdc}(h2, N) = 1$, ou seja, a sondagem visita todas as N posicoes antes de repetir.

4.4 Como a insercao conta colisoes

Na funcao insere(), o loop começa em $i=0$. Se a posicao inicial ja ta ocupada, i sobe pra 1, 2, etc. Quando acha um slot livre, retorna i, que e exatamente o numero de slots cheios visitados antes (as colisoes dessa insercao). O main acumula esses valores e imprime a cada 100 insercoes.

4.5 Saida do programa

Stderr:

```
CPFs lidos: 4096 tamanho da tabela: 5003
```

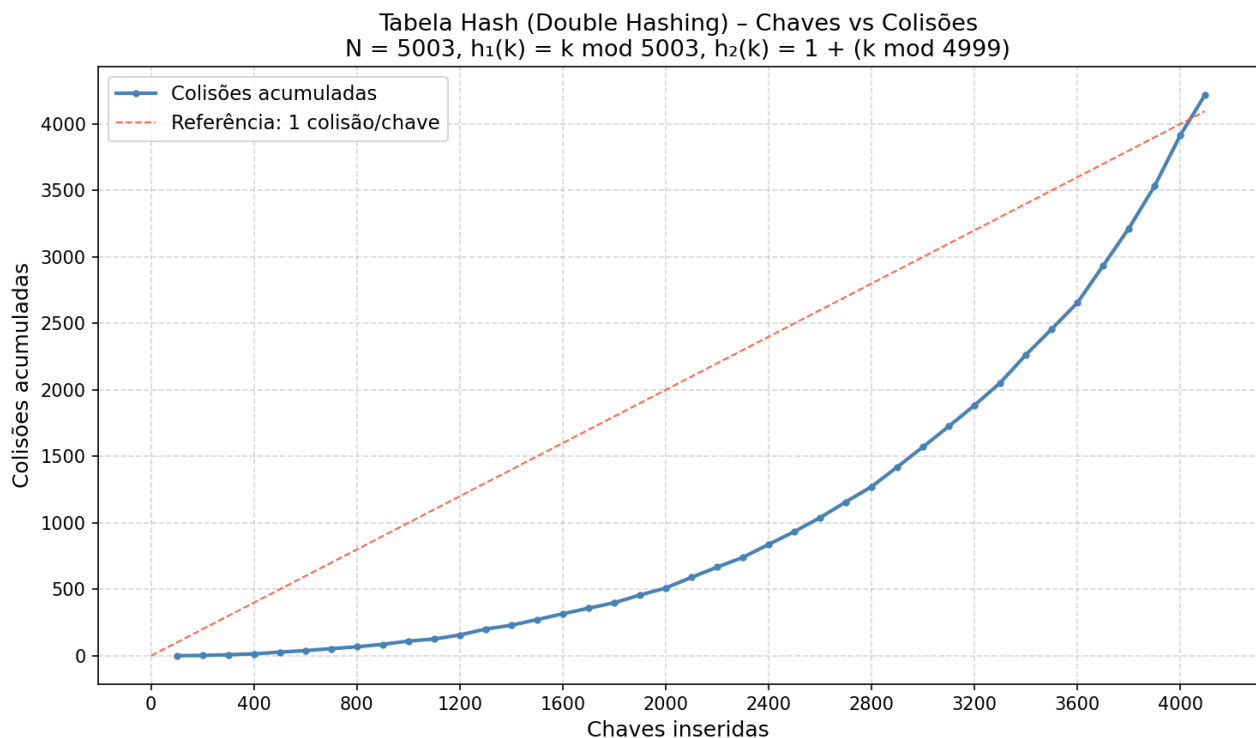
Stdout (CSV):

```
chaves_inseridas,colisoes_acumuladas
100,1
200,3
300,8
400,15
500,28
600,39
700,54
800,68
900,86
1000,111
1100,126
1200,157
1300,201
1400,230
1500,272
1600,316
1700,358
1800,399
1900,457
2000,510
2100,590
2200,667
2300,740
2400,838
```

2500, 934
 2600, 1039
 2700, 1157
 2800, 1272
 2900, 1420
 3000, 1571
 3100, 1725
 3200, 1884
 3300, 2053
 3400, 2264
 3500, 2457
 3600, 2655
 3700, 2934
 3800, 3215
 3900, 3533
 4000, 3915
 4096, 4219

4.6 Grafico

O grafico mostra que ate umas 2000 chaves as colisoes crescem quase linear. Depois acelera bastante quando a tabela passa dos 60% de ocupacao. Com todas as 4096 chaves o total foi 4219 colisoes, razoavel pra um fator de carga de ~82%.



5. Observacoes e Conclusoes

Como compilar e rodar

```
gcc hash_cpf.c -o hash_cpf -O2
```

Windows:

```
.\hash_cpf.exe > resultados.csv

# Linux:
./hash_cpf > resultados.csv

# grafico (pip install matplotlib se nao tiver):
python grafico.py resultados.csv
```

O que funciona

- Le os 4096 CPFs corretamente.
- Insere todos sem a tabela ficar cheia.
- Conta as colisoes certo.
- Gera CSV e grafico.

Limitacoes

- So faz insercao, nao tem busca nem remocao.
- Tabela com tamanho fixo, nao redimensiona.
- CPF duplicado no arquivo e ignorado silenciosamente.

Dificuldades

Cometi um erro bobo no inicio: o fscanf tava com dois campos ("%d %lld"), entao ele consumia um token extra por iteracao e eu so lia 2048 CPFs em vez de 4096. Corrigi pra "%lld" simples.

Outro problema foi que o PowerShell salva o stdout em UTF-16 LE com BOM, e o Python nao conseguia ler o CSV. Resolvi passando encoding="utf-16" no open() do grafico.py.

Conclusao

O double hashing funcionou bem. O grafico confirma o que a teoria diz: colisoes crescem devagar no começo e aceleram quando a tabela enche. Escolher N primo foi essencial pra o passo do double hashing cobrir a tabela inteira.